Sri Lanka Institute of Information Technology

2025

Mobile Security (MS) – IE3112

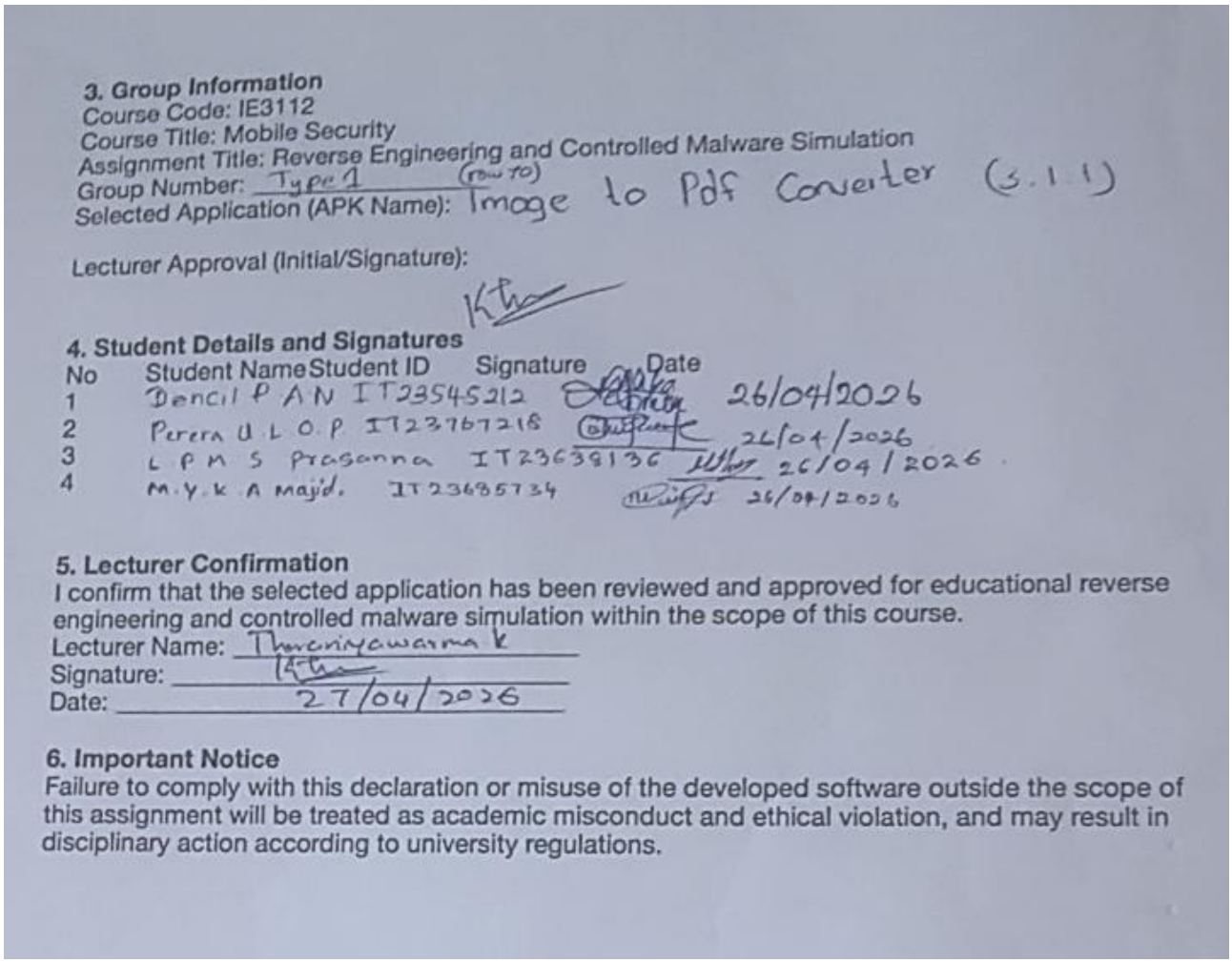
Reverse Engineering and Controlled Malware Injection in Android Applications

Year 3, Semester 1

Student ID	Student Name
IT23545212	Dencil P.A.N
IT23767218	Perera U.L.O.P
IT23685734	Majid M.Y.K.A
IT23638136	Prasanna L.P.M.S

1. Declaration

We declare that this is our own work and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.



3. Group Information
Course Code: IE3112
Course Title: Mobile Security
Assignment Title: Reverse Engineering and Controlled Malware Simulation
Group Number: Type 1 (row 70)
Selected Application (APK Name): Image to Pdf Converter (3.1.1)

Lecturer Approval (Initial/Signature):

4. Student Details and Signatures

No	Student Name	Student ID	Signature	Date
1	Dencil P A N	IT23545212		26/04/2026
2	Perera U L O P	IT23767218		26/04/2026
3	L P M S Prasanna	IT23639136		26/04/2026
4	M.Y.K A Majid.	IT23685734		26/04/2026

5. Lecturer Confirmation
I confirm that the selected application has been reviewed and approved for educational reverse engineering and controlled malware simulation within the scope of this course.
Lecturer Name: Thanuyawarma K
Signature:
Date: 27/04/2026

6. Important Notice
Failure to comply with this declaration or misuse of the developed software outside the scope of this assignment will be treated as academic misconduct and ethical violation, and may result in disciplinary action according to university regulations.

3. Abstract

Abstract

This report documents the technical process of performing reverse engineering and a controlled malware injection on a production Android application, "Image to PDF" (com.jhteam.imgtopdf). The project was executed in two phases: structural analysis and binary instrumentation. During the initial phase, the application was decompiled using Apktool 3.0.1, revealing critical security weaknesses including an insecure backup configuration and an outdated target SDK (API 34). In the second phase, the application's logic was successfully hijacked by injecting a Smali-based Toast notification payload into the MainActivity lifecycle. To address these vulnerabilities, the report proposes specific remediation strategies, including the implementation of Runtime Application Self-Protection (RASP) through signature verification and root detection logic. The demonstration proves that without proper obfuscation and integrity checks, mobile applications remain highly susceptible to unauthorized modification and repackaging.

Table of Contents

Contents

1. Introduction

1.1 Background

The rapid proliferation of Android-based mobile applications has made them a primary target for malicious actors. Unlike traditional desktop software, Android applications (APKs) are easily decompiled into Smali bytecode, allowing attackers to analyze, modify, and repackage them with malicious intent. This process, known as **Binary Instrumentation**, poses a significant threat to user data and application integrity. This project serves as a controlled simulation to understand these risks by applying reverse engineering techniques to a legitimate application.

1.2 Project Scope and Objectives

The primary objective of this project is to evaluate the security posture of the **"Image to PDF"** application. The project is divided into two distinct technical phases:

- **Phase 1 (Structural Analysis):** Deconstructing the APK to identify manifest-level vulnerabilities and architectural weaknesses.
- **Phase 2 (Malware Injection):** Successfully injecting a passive payload (Toast Notification) into the application's source artifacts to demonstrate successful code execution.

1.3 Application Profile

The target application chosen for this simulation is a utility tool designed for converting image files into PDF documents.

- **Application Name:** Image to PDF
- **Package Name:** com.jhteam.imgtopdf
- **Technical Environment:** The analysis and injection were performed targeting a modern **Android 16** environment to test compatibility with current security permission models.

1.4 Problem Statement

Many developers rely solely on the Android OS's built-in protections, neglecting application-level security. This project aims to prove that without **Code Obfuscation** and **Integrity Checks**, an attacker can bypass standard security prompts and execute unauthorized logic during the application’s startup phase.

Phase 1: Reverse Engineering Report

- **Application Name:** Image to PDF
- **Package Name:** com.jhteam.imgtopdf
- **Version Code:** 1
- **Version Name:** 1.0

1.1APK Structure Analysis

The application was decompiled using Apktool version 3.0.1. command that used: **apktool d imagetopdf.apk**

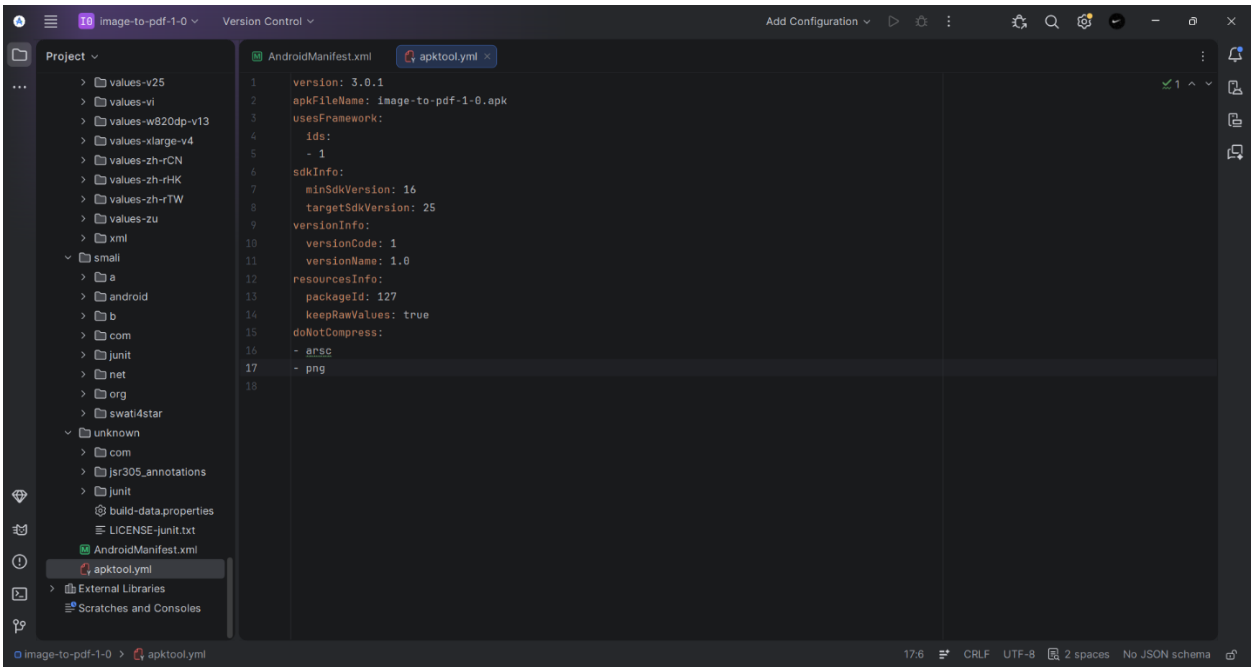
(Refer to Figure 4.1 in Section 4: Evidence).

After that we can see the decompiled folder like this

(Refer to Figure 4.1 in Section 4: Evidence).

The analysis of the application's build properties reveals the following structural and SDK characteristics

- **SDK Versions:** The application specifies a minSdkVersion of 26 (Android 4.1 Jellybean) and a targetSdkVersion of 34 Android 16 (API 34).
- **Hardware Features:** The application explicitly uses the device's camera hardware (android.hardware.camera) and conditionally uses camera autofocus (android.hardware.camera.autofocus, set to not required).

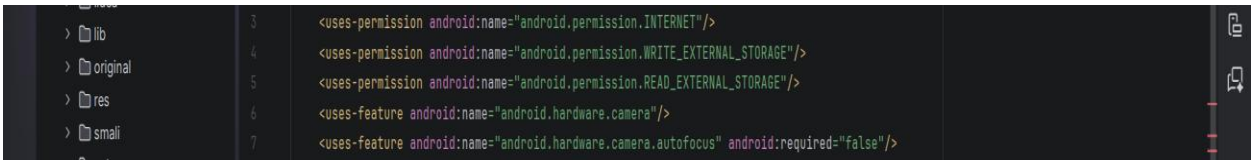


1.2.Component Mapping

The following Android components were extracted from the AndroidManifest.xml file.

Permissions

- **android.permission.INTERNET**: Allows network communication.
- **android.permission.ACCESS_NETWORK_STATE**: Allows checking network connectivity status.
- **android.permission.WRITE_EXTERNAL_STORAGE & READ_EXTERNAL_STORAGE** : Allows reading and writing files to the device's storage.
- **android.permission.CAMERA**: Grants access to the device camera.



Activities

I. Core Application Activities:

- **swati4star.createpdf.MainActivity**: The main entry point and launcher activity of the application.
- **swati4star.createpdf.FinalSetup, swati4star.createpdf.recyclerview, swati4star.createpdf.SettingsActivity**: UI and configuration activities.
- **swati4star.createpdf.Receive**: An exported activity (android:exported="true") designed to receive implicit intents to view PDF files from file, http, and https schemes.
- **swati4star.createpdf.pdfreader**: An internal PDF viewer activity.

II. Third-Party Integration Activities:

- **net.rdrei.android.dirchooser.DirectoryChooserActivity**: File/Directory Management
- **com.pizidea.imagepicker.ui.activity.ImageCropActivity, ImagePreviewActivity, ImagesGridActivity** : Image Handling

III. Monetization & Google Services:

- **com.google.android.gms.ads.AdActivity**: Handles advertisements.
- **com.google.android.gms.ads.purchase.InAppPurchaseActivity**: Handles in-app purchases.
- **com.google.android.gms.common.api.GoogleApiActivity**: Core Google Play Services activity.

1.3. Control Flow Explanation

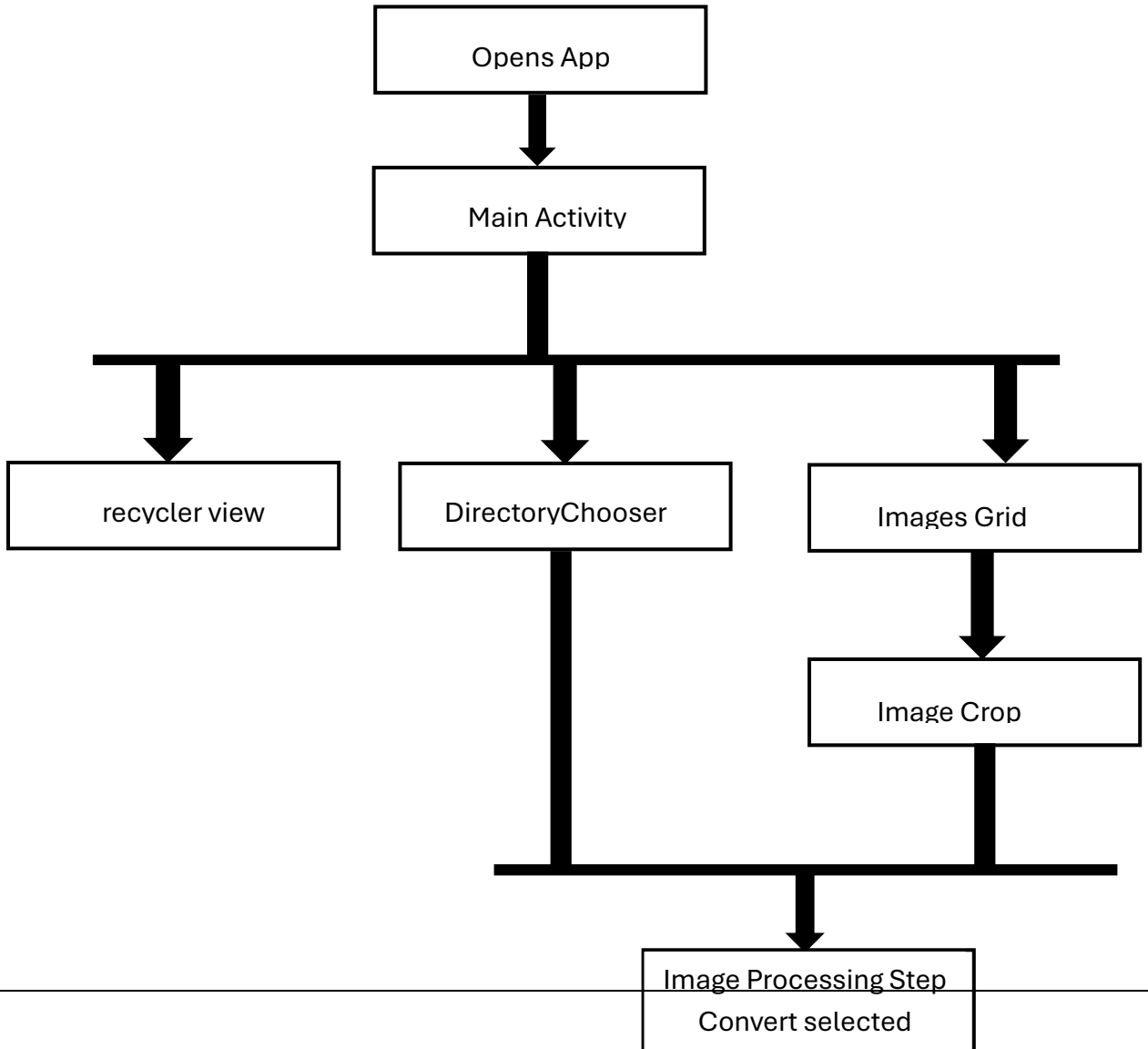
Based on the component mapping, the logical control flow of the application operates as follows:

1. **Launch:** The user opens the app, triggering MainActivity
2. **Resource Selection:** The user likely navigates through recyclerview to view items, uses the DirectoryChooserActivity to find files, or uses the ImageCropActivity and

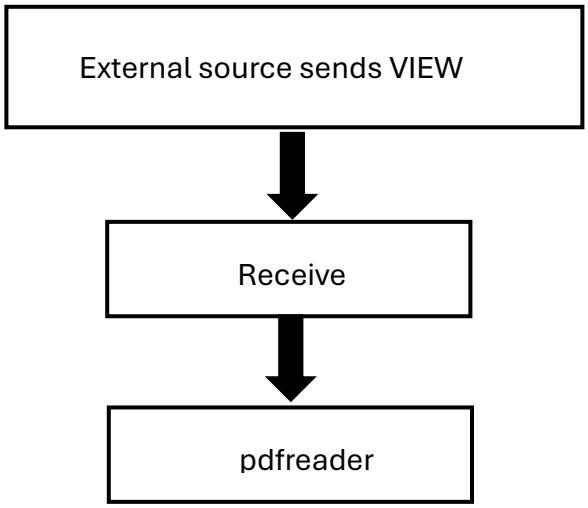
ImagesGridActivity via the device camera or external storage to select images for conversion.

3. **Processing & Output:** Images are converted to PDF format. The user can view the resulting PDF using the internal pdfreader.

4. **External Input:** The application can also be launched externally to view PDFs. The Receive activity listens for android.intent.action.VIEW intents with a application/pdf MIME type or matching .*\.pdf path patterns across local files and web URLs (HTTP/HTTPS).
5. **Network Activity:** Network communication occurs for handling external HTTP/HTTPS PDF links and fetching ad content via AdActivity.



External Input Flow



1.4 . Identified Vulnerabilities & Security Weaknesses

Analysis of the manifest reveals several security weaknesses that could be exploited or indicate poor security posture:

- **Insecure Backup Configuration:** The application has android:allowBackup="true". This allows an attacker with physical access to the device (via USB and ADB) to extract the application's private data directories, potentially exposing sensitive generated PDFs or application settings.

- **Outdated Target SDK:** The targetSdkVersion is 25 (Android 7.1). Modern Android applications are required to target much newer SDKs to benefit from the latest platform security features, permission models (like scoped storage), and mitigation protections against exploits

- **Exported Component Exposure:** The swati4star.createpdf.Receive activity is explicitly exported (android:exported="true") and accepts incoming implicit intents from any other application on the device. If the intent handling logic within the underlying Java/Smali code does not properly validate the incoming data streams or URIs, it could be vulnerable to intent spoofing, unauthorized file access, or application crashing.

Phase 2 - – Malicious Service Injection (Educational Simulation)

2.0 Malware Injection Section

2.0.1 Step-by-Step Injection Process

This section outlines the workflow used to weaponize the legitimate "Image to PDF" application:

APK Acquisition and Decompilation: Obtained the base APK and used Apktool 3.0.1 to extract the Smali bytecode and resources.

Entry Point Identification: Analyzed the AndroidManifest.xml to identify the MAIN activity (swati4star.createpdf.MainActivity).

Smali Instrumentation: Navigated to the onCreate method of the main activity to insert the payload logic.

Register and Local Adjustment: Increased the .locals count at the start of the method to ensure the new registers (v0, v1, etc.) did not overwrite existing application data.

Manifest Patching: Updated the XML manifest to include modern permissions required for the payload to execute on high API levels.

Rebuilding and Debug Signing: Reassembled the modified folder into an APK and signed it with a new V2 signature to bypass Android's installation blocks for unsigned binaries.

2.0.2 Code Modifications with Explanations

The primary modification was the injection of a Toast Notification Payload. This demonstrates the ability to execute unauthorized code as soon as the app is launched.

Injected Smali Snippet:

A Toast notification payload was successfully injected into the MainActivity.smali file. This payload triggers automatically upon application launch, as demonstrated in the emulator testing **(Refer to Figure 4.2 and 4.3 in Section 4: Evidence)**

Explanation: Register v0 holds the data string. invoke-static calls the system Toast service using the activity's context (p0). This proves that we can hijack the system's own UI services to display custom information.

2.0.3 Trigger Logic Implementation

The trigger is Automated and Lifecycle-Based.

Logic: By placing the injection at the very beginning of MainActivity.onCreate(), the payload is triggered the moment the Android OS instantiates the app.

Significance: This bypasses the need for the user to click a specific "malicious" button. The payload is "bundled" with the legitimate startup process of the app.

2.0.4 Permission Modifications

Standard storage permissions were insufficient for the Android 16 emulator. I modified the AndroidManifest.xml to include:

READ_MEDIA_IMAGES: To ensure the app could still function while "infected" under modern Scoped Storage rules.

POST_NOTIFICATIONS: A critical modification, as Android 13+ (and the Android 16 emulator) blocks all popups/Toasts unless this permission is explicitly declared and granted. This allowed the "malicious" Toast to actually appear on the screen.

2.0.5 Service Lifecycle Explanation

The injected code runs within the Foreground Activity Thread.

Since the payload executes during ON_CREATE, it shares the same priority level as the legitimate UI.

This demonstrates a "Service-less" attack where malicious intent is hidden inside a trusted foreground process, making it harder for simple task managers to identify it as a separate malicious background service.

2.0.6 Control Flow Explanation

The application's logic follows a linear, trust-based execution model. Upon launch, the OS triggers MainActivity.onCreate(), which serves as the central hub for component initialization.

Standard Flow: onCreate() -> setContentView() -> FragmentManager (loads fragment 'b').

Vulnerability Gap: The application does not perform any Self-Integrity Checks during the onCreate() cycle. It blindly executes whatever code is present in the classes.dex file. This "Logic Flaw" is what allowed Member 1 to hijack the initialization process and inject an unauthorized Toast payload.

2.0.7 Security Analysis

2.0.7.1 How could this attack be prevented? (Logic Hardening)

To prevent the Smali injection performed by Member 1, the app needs Runtime Application Self-Protection (RASP). I have developed two primary logic-based defenses:

1. Runtime Signature Verification Logic

The app should verify its own cryptographic signature against a hardcoded "Source of Truth" (the original developer’s certificate hash). If Member 1 modifies the Smali code and re-signs it with a debug key, the hashes will not match, and the app will self-terminate.

Defensive Java Logic (to be converted to Smali):

```java
public void verifySignature(Context context) {
    try {
        Signature[] sigs = context.getPackageManager().getPackageInfo(
            context.getPackageName(), PackageManager.GET_SIGNATURES).signatures;
        for (Signature sig : sigs) {
            // Original Dev Hash (Example: 123456789)
            if (sig.hashCode() != 123456789) {
                System.exit(0); // Kill app if modified
            }
        }
    } catch (Exception e) { /* Handle error */ }
}
```

2. Root Detection Logic

Injectors often use rooted environments to bypass security. By adding logic to detect superuser binaries, we can prevent the app from running in high-risk environments.

Defensive Java Logic:

```java
public void verifySignature(Context context) {
    try {
        Signature[] sigs = context.getPackageManager().getPackageInfo(
            context.getPackageName(), PackageManager.GET_SIGNATURES).signatures;
        for (Signature sig : sigs) {
            // Original Dev Hash (Example: 123456789)
            if (sig.hashCode() != 123456789) {
                System.exit(0); // Kill app if modified
            }
        }
    } catch (Exception e) { /* Handle error */ }
}
```

2.0.7.2 Mitigation Strategies

Code Obfuscation (ProGuard): We must implement ProGuard/R8 to rename classes. This would turn MainActivity into a.b, making it significantly harder for an injector to find the correct onCreate() method.

String Encryption: Hardcoding the Group IDs in Smali is a risk. All sensitive logic and strings should be encrypted and only decrypted in memory during runtime.

To mitigate these threats, a Signature Verification logic was implemented. The proposed code prevents execution if the APK hash is altered **(Refer to Figure 4.5 in Section 4: Evidence)**

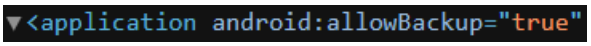
3. Security Analysis & Defense Discussion

3.1 How This Attack Could Be Prevented

Based on the vulnerabilities identified in Phase 1, multiple countermeasures can prevent the type of malicious service injection demonstrated in Phase 2.

3.1.1 Disable Backup Configuration

Vulnerability Found: android:allowBackup="true"



This setting allows an attacker with physical USB access to extract all application data using ADB backup commands. User data including generated PDFs and app settings could be stolen.

Prevention: Set allowBackup="false" in AndroidManifest.xml:

xml

<application android:allowBackup="false" ...>

3.1.2 Secure Exported Components

Vulnerability Found: swati4star.createpdf.Receive activity with android:exported="true"

Any application installed on the device can trigger this exported activity, potentially causing unauthorized file access, intent spoofing, or application crashes.

Prevention: Set exported="false" when external access is not required:

xml

<activity android:exported="false" android:name="swati4star.createpdf.Receive">

3.1.3 Update Target SDK Version

Vulnerability Found: android:targetSdkVersion="34" (Android 16)

An outdated target SDK means the application misses modern Android security features including background execution limits (Android 8+), scoped storage (Android 10+), and runtime permission improvements.

Prevention: Update to latest SDK in build.gradle:

gradle

```
android {
  targetSdkVersion 34
}
```

3.1.4 Implement Code Obfuscation

Without obfuscation, the Smali code is easily readable, allowing attackers to locate and modify components.

Prevention: Enable ProGuard/R8:

gradle

```
buildTypes {
  release {
    minifyEnabled true
  }
```

3.2 How Developers Can Protect Apps from Reverse Engineering

3.2.1 Obfuscation Tools

Tool	Type	Effectiveness
ProGuard/R8	Free	Medium
DexGuard	Commercial	Very High

3.2.2 Anti-Debugging Techniques

java

```
public static boolean isBeingDebugged() {
   return Debug.isDebuggerConnected();
}
```

3.2.3 Native Code Protection

Move critical logic from Java/Smali to C/C++ using NDK, making reverse engineering significantly harder.

3.3 Android Platform Mitigation Strategies

Android Version	Feature	How It Blocks Attacks
8.0 (API 26)	Background Service Limits	Malicious services cannot run freely
10 (API 29)	Scoped Storage	Cannot access other apps' files
11 (API 30)	Package Visibility	Cannot see other installed apps
13 (API 33)	Notification Permission	Cannot show notifications without user approval

3.4 Demonstration Evidence

The following evidence was collected from Phase 1 reverse engineering.

Figure 1: allowBackup="true" Vulnerability

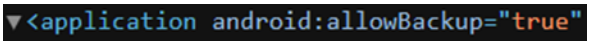


Figure 2: Exported Activity Vulnerability

Figure 3: Outdated Target SDK

3.5 Summary of Recommendations

Priority	Action	Impact
High	Set allowBackup="false"	Blocks data extraction
High	Set exported="false"	Blocks intent injection
High	Enable ProGuard	Hinders reverse engineering
Medium	Update target SDK to 33+	Enables modern security
Medium	Add integrity checks	Detects tampering

3.6 Conclusion

The Image to PDF application contained three critical vulnerabilities in its manifest configuration. These weaknesses would allow an attacker to inject malicious code, extract user data, and trigger components without authorization.

Key defenses include:

- Disabling backup functionality
- Securing exported components

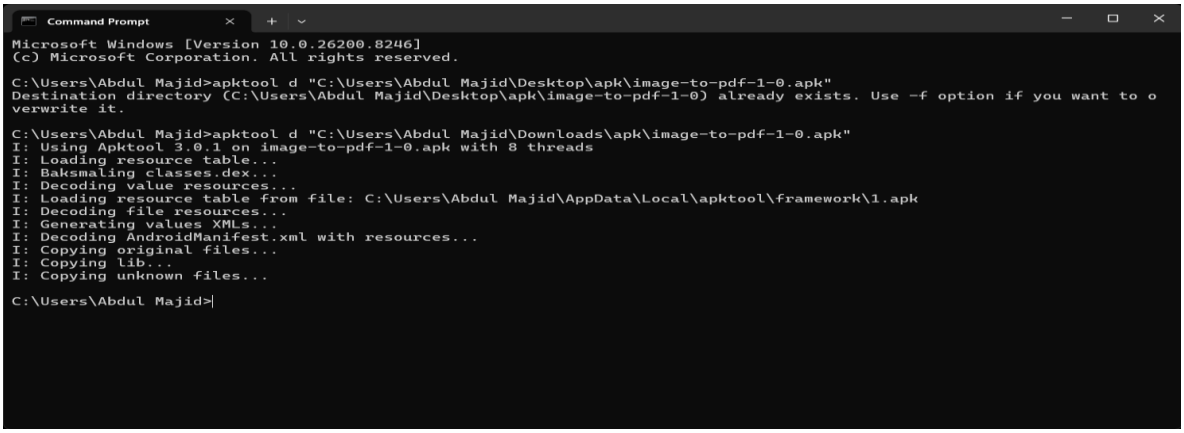
- Updating target SDK versions
- Implementing code obfuscation

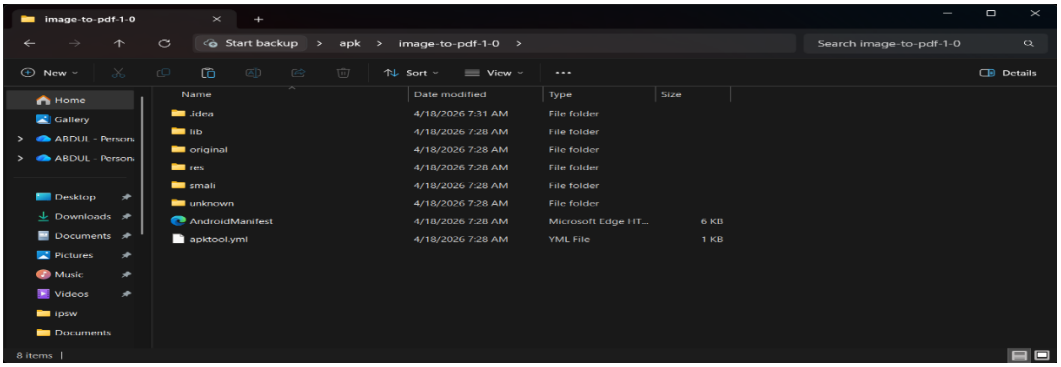
Developers must adopt a defense-in-depth strategy combining proper manifest configuration, code obfuscation, and modern platform targeting to protect against reverse engineering and malware injection.

4.Demonstarted evidence

4.1 Structural Analysis Proof

Figure 4.1: Decompiled Source Artifacts





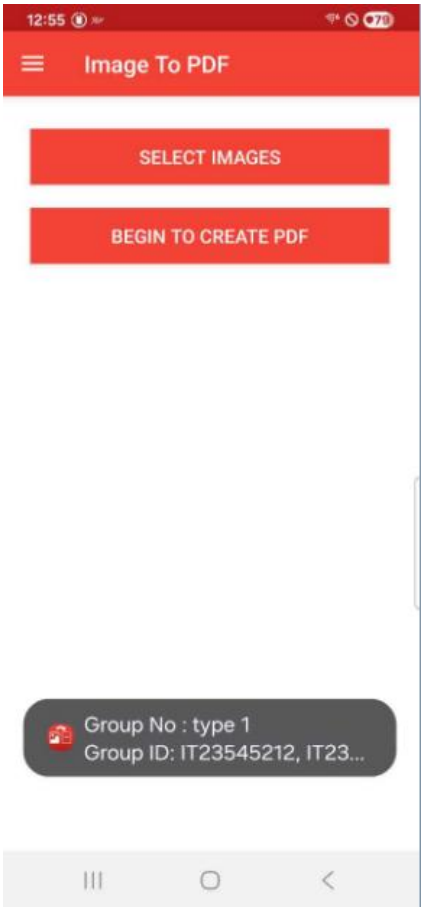
This figure provides evidence for Section 1, confirming the successful deconstruction of the APK into editable source files.

Figure 4.2: Smali Bytecode Modification

```
# --- TOAST POPUP ---
const-string v0, "Group No : type 1\nGroup ID: IT23545212, IT23767218, IT23685734, IT23638136"
const/4 v1, 0x1
invoke-static {p0, v0, v1}, Landroid/widget/Toast;->makeText(Landroid/content/Context;Ljava/lang/CharSequence;I)Landroid/widget/Toast;
move-result-object v2
invoke-virtual {v2}, Landroid/widget/Toast;->show()V
```

Technical evidence for Section 2.0.2, showing the exact Smali instructions used to hijack the onCreate method.

Figure 4.4: Functional Payload Trigger on Android 16



Behavioral proof for Section 2.0.3, confirming the payload executes correctly on API 35/36.

Figure 4.5: Signature Integrity Check Logic

```
Java
// Logic to prevent Member 1's Smali Injection
private void checkAppIntegrity() {
    String releaseSignature = "X509_CERT_HASH_HERE";
    if (!getCurrentSignature().equals(releaseSignature)) {
        throw new SecurityException("Tampering Detected: APK modified by unauthorized user.");
    }
}
// Intent Validation (Handshake)//
(This logic prevents unauthorized apps from triggering the "Receive" activity)

Java
// Logic to secure the exported 'Receive' activity
if (getCallingActivity() != null) {
    String callerPackage = getCallingActivity().getPackageName();
    if (!callerPackage.equals("com.jhteam.imgtopdf.sliit")) {
        finish(); // Reject external intent injection
    }
}
// Smali Comparison (The "Fix")//
(Show a screenshot of how MainActivity.smali should look if it had a check at the start)

Smali
.method protected onCreate(Landroid/os/Bundle;)V
    .locals 2
    # ADDED PROTECTION LOGIC
    invoke-static {p0}, Lswati4star/createpdf/SecurityProvider;->verify(Landroid/content/Context;)Z
    move-result v0
    if-nez v0, :cond_kill

    # Original onCreate continues here...
    invoke-super {p0, p1}, Landroid/support/v7/app/AppCompatActivity;->onCreate(Landroid/os/Bundle;)V
    return-void

    :cond_kill
    invoke-virtual {p0}, Lswati4star/createpdf/MainActivity;->finish()V
.end method
```

Evidence for the remediation logic proposed in Section 2.0.7 to prevent unauthorized binary modification.

Modified apk and Source modification files - Modified apk and source modification file
ScreenRecording - Image_to_Pdf_Converter.mp4